TERRAFORM WORKSPACE

A **Terraform Workspace** is an isolated environment within a single Terraform configuration, allowing you to manage multiple instances of the same infrastructure without duplicating code.
Each workspace has its own **state file**, which keeps track of the resources managed by Terraform in that specific workspace.

Uses of Terraform Workspaces:

1. **Environment Isolation:**
   Use workspaces to manage different environments (e.g., dev, staging, prod) with the same configuration.

2. **Multi-Tenant Applications:**
   Manage separate infrastructure for different clients or tenants.

3. **Avoid State File Conflicts:**
   Keep state files isolated for better organization and avoid conflicts when managing resources.

How Workspaces Work:

1. The default workspace is called default.
2. You can create, switch, and manage workspaces using Terraform commands.
3. Each workspace has its own state file stored in the backend.

Key Commands:
List all workspaces: `terraform workspace list`
Create a new workspace: `terraform workspace new <workspace_name>`

Switch to a workspace:

```
terraform workspace select <workspace_name>
```

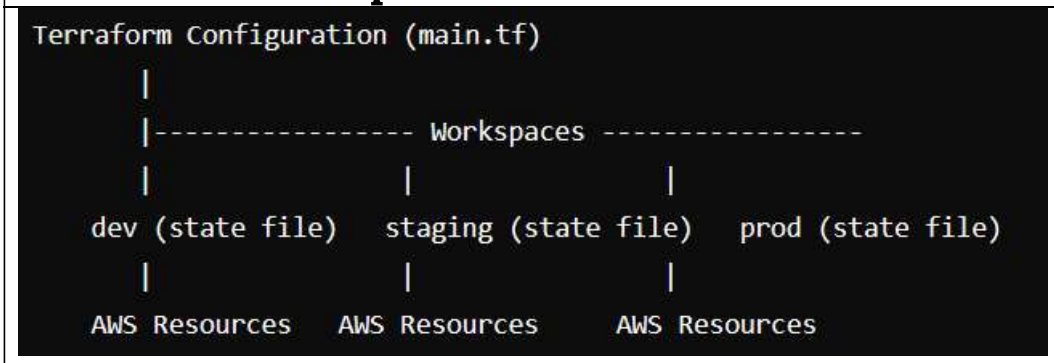
Delete a workspace (if empty):

```
terraform workspace delete <workspace_name>
```

Diagram: Terraform Workspaces Example

Here's a simple diagram explaining workspaces:

Terraform Workspaces for Environment Isolation:



- **dev:** Contains resources for development (e.g., small EC2 instances).
- **staging:** Contains resources for testing.
- **prod:** Contains production-grade resources.

Each workspace manages its own **state file** and resources independently, ensuring no overlap.

Command Flow Example:

```
terraform workspace new dev
terraform apply -var-file=dev.tfvars

terraform workspace new staging
terraform apply -var-file=staging.tfvars

terraform workspace new prod
terraform apply -var-file=prod.tfvars
```

For this demo we will be using the previous terraform folder named “tf-import-s3”

```
PS D:\Downloads\TERRAFORM\AWS\tf-import-s3> terraform workspace list
* default
```

```
PS D:\Downloads\TERRAFORM\AWS\tf-import-s3> terraform workspace new dev
Created and switched to workspace "dev"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS D:\Downloads\TERRAFORM\AWS\tf-import-s3>
```

When you apply a Terraform configuration across multiple workspaces, creating resources with the same name (like an S3 bucket) in each workspace will cause an error, as most resources (e.g., AWS S3 buckets) require unique names globally.

To resolve this issue, you can use the terraform.workspace interpolation to include the workspace name in the resource's name, ensuring it is unique for each environment.

Something like this

```
resource "aws_s3_bucket" "example" {
  bucket = "my-app-bucket-${terraform.workspace}"
  acl    = "private"
}
```

Sample of fi